Machine Learning — Core Theory Notes

Regression | Classification | Regularisation | Metrics | Cross-Validation

Q1 What is Regression? How is it different from Classification?

Definition of Regression:

Regression is a supervised machine learning technique used to predict a **continuous numerical output** (a real-valued quantity). The model learns a mathematical mapping from input features X to a continuous target variable y by minimising a loss function — typically Mean Squared Error (MSE).

```
Goal: learn f(X) such that y_hat = f(X) minimises MSE = (1/n) * SUM[ (y_i - y_hat_i)^2 ]
```

Definition of Classification:

Classification predicts a **discrete class label** (a category) from a finite set. The model learns decision boundaries that separate data points into distinct groups. Loss is typically Cross-Entropy.

Key Distinction at a Glance:

- **Regression** → Output is a NUMBER (salary, temperature, house price)
- **Classification** → Output is a CATEGORY (spam/not spam, disease/healthy)

Comparison Table:

Aspect	Regression	Classification
Output Type	Continuous number	Discrete class label
Example Output	■68,40,000 property price	APPROVED / REJECTED
Loss Function	MSE / RMSE / MAE	Cross-Entropy / Log Loss
Evaluation	R^2, RMSE, MAE	Accuracy, Precision, Recall, F1
Decision Boundary	No fixed boundary — learns a curve	Learns a separating hyperplane
Algorithms	Linear Reg., Ridge, XGBoost Reg.	Logistic Reg., SVM, RF Classifier

Real-World Indian Examples:

Example 1 — HDFC Bank: EMI Amount Prediction (Regression)

HDFC's home loan engine collects applicant features — monthly income, CIBIL credit score, loan tenor, property valuation, LTV ratio — and predicts the exact monthly EMI (a continuous number in ■). Since EMI can be ■12,500 or ■87,430.50, this is a regression problem where the model minimises prediction error on a continuous scale.

Features (X): Monthly Income, CIBIL Score, Loan Amount, Tenor, Age

Target (y): EMI in ₹ (e.g., ₹24,350.75/month)

Example 2 — Maharashtra Agriculture: Crop Yield Prediction (Regression)

The Maharashtra Government and ICAR use satellite remote-sensing data, soil health cards, rainfall records, and fertiliser usage to predict wheat/soybean yield in quintals/hectare for Vidarbha and Marathwada farmers. This continuous output guides MSP (Minimum Support Price) procurement and mandi supply planning.

Features (X): Rainfall (mm), Soil NPK, Temperature, Sowing Date, Crop Variety

Target (y): Yield in quintals/hectare (e.g., 18.6 q/ha)

Counter-example (Classification): *SBI's loan default detection. Here the output is binary — DEFAULT (1) or NO DEFAULT (0) — so Classification models like Logistic Regression or XGBoost Classifier are used, not regression.*

Q2 Simple Linear Regression vs Multiple Linear Regression vs Regularised Regression

Simple Linear Regression (SLR):

Models the linear relationship between **one independent variable (feature)** and one dependent variable (target). It finds the best-fit straight line through the data.

```
y = beta_0 + beta_1 * X + epsilon

Where: beta_0 = intercept, beta_1 = slope, epsilon = error term
```

Example: Predicting a student's exam marks (y) purely based on hours studied (X).

Limitation: *Real-world outcomes rarely depend on just one variable.*

Multiple Linear Regression (MLR):

Extends SLR to **two or more independent variables**. The model fits a hyperplane (multi-dimensional surface) through the data points.

```
y = beta_0 + beta_1*X_1 + beta_2*X_2 + ... + beta_p*X_p + epsilon

Example: SalePrice = b0 + b1*GrLivArea + b2*OverallQual + b3*YearBuilt + ...
```

Example: HDFC property valuation model — SalePrice predicted using 79 features (area, quality, age, garage, neighbourhood etc.) — exactly the Ames Housing problem.

Problem: *When features are many (p >> n) or strongly correlated (multicollinearity), OLS coefficient estimates become* ***unstable and blow up*** *— overfitting occurs.*

Regularised Regression (Ridge & Lasso):

Regularisation adds a **penalty term** to the OLS loss function that shrinks large coefficients toward zero. This reduces variance (overfitting) at the cost of a small increase in bias — the classic Bias-Variance Trade-off.

```
Ridge Loss = SUM[(y_i - y_hat_i)^2] + lambda * SUM[ beta_j^2 ] (L2 penalty)

Lasso Loss = SUM[(y_i - y_hat_i)^2] + lambda * SUM[ |beta_j| ] (L1 penalty)
```

Property	Ridge (L2)	Lasso (L1)
Penalty	Square of coefficients (L2 norm)	Absolute value of coefficients (L1 norm)
Coefficient Behaviour	Shrinks all toward zero; none become exactly 0	Can force some to exactly 0 (feature selection)
Sparsity	Dense — all features retained	Sparse — irrelevant features eliminated
Best For	Multicollinearity; many small effects	High-dimensional data; automatic feature selection
Alpha Tuning	RidgeCV auto-selects optimal alpha	LassoCV auto-selects optimal alpha
Our Project Result	RMSE $20,497 (alpha=10)	RMSE $20,058 (alpha=0.001, 123 features zeroed)

Key Insight: *When lambda = 0, regularised regression reduces to plain OLS. As lambda increases, model coefficients shrink more aggressively, trading lower variance for higher bias.*

Q3 Explain Overfitting and Underfitting. How does Regularisation (L1/L2) help?

Underfitting (High Bias):

The model is too simple to capture the true relationship in the data. It performs poorly on **both training and test sets**. Analogously — a student who did not study enough fails both mock exams and the final.

Characteristic	Underfitting	Overfitting
Also Called	High Bias Model	High Variance Model
Training Error	HIGH	Very LOW (memorises data)
Test Error	HIGH	HIGH (fails to generalise)
Root Cause	Model too simple / few features	Model too complex / too many features
Visual Sign	Flat/straight line through curved data	Wiggly curve that passes through every data point
Fix	More features, higher-order terms, more complex model	Regularisation, more data, cross-validation, dropout

Overfitting (High Variance):

The model memorises training data noise and random fluctuations instead of learning the true underlying pattern. It performs **excellently on training data but poorly on unseen test data**. Like a student who mugged only previous year questions — helpless when a new question pattern appears in the final exam.

The Bias-Variance Trade-Off:

```
Total Error = Bias^2 + Variance + Irreducible Noise

Goal: minimise BOTH bias AND variance simultaneously
```

How L1 (Lasso) and L2 (Ridge) Regularisation Help:

Mechanism: Regularisation modifies the objective function by adding a penalty on coefficient magnitudes. This forces the optimiser to keep weights small, preventing the model from fitting noise.

Ridge (L2): Penalises large squared coefficients → spreads weight evenly across correlated features → reduces coefficient instability without eliminating any feature.

Lasso (L1): Penalises absolute values → geometrically, the L1 ball has corners on the axes, so the optimal solution often lands exactly on an axis (coefficient = 0) → automatic feature selection.

Lambda (alpha) Effect:

• lambda = 0 → Plain OLS regression (may overfit)

• lambda = small → Mild regularisation (sweet spot)

• lambda = large → All coefficients → 0 (underfitting)

• Optimal lambda found via Cross-Validation (RidgeCV / LassoCV)

Our Project Evidence: *Plain Linear Regression (no regularisation) achieved RMSE = $22,630 on the Ames Housing test set. Adding Ridge regularisation improved RMSE to $20,497. Lasso further improved to $20,058 by pruning 123 redundant features.*

Q4 Define RMSE, MAE, and R². When would you prefer RMSE over MAE?

Mean Absolute Error (MAE):

```
MAE = (1/n) * SUM[ |y_i - y_hat_i| ]
```

• Measures the **average absolute deviation** between actual and predicted values.

• **Robust to outliers** — each error contributes equally regardless of its magnitude.

• Interpretable in original units (e.g., 'on average predictions are off by ■14,162').

• Gradient is constant → can cause oscillation near the minimum during gradient descent.

Root Mean Squared Error (RMSE):

```
RMSE = SQRT[ (1/n) * SUM[ (y_i - y_hat_i)^2 ] ]
```

• **Squares errors** before averaging, then takes the square root → larger errors are penalised more heavily than smaller ones.

• Sensitive to outliers and large deviations.

• Also interpretable in original units (same scale as y).

• RMSE >= MAE always. When RMSE >> MAE, large outlier errors are present in the predictions.

R-Squared (Coefficient of Determination, R^2):

```
R^2 = 1 - [ SUM(y_i - y_hat_i)^2 / SUM(y_i - y_mean)^2 ]

Interpretation: R^2 = 0.93 means model explains 93% of variance in SalePrice
```

• **Scale-free** measure between 0 and 1 (can be negative for very poor models).

• $R^2 = 1.0$ → perfect predictions. $R^2 = 0$ → model is no better than predicting the mean.

• Does not penalise for model complexity → use Adjusted R^2 when comparing models with different number of features.

RMSE vs MAE — When to Prefer Which?

Scenario	Preferred Metric	Reason
Large outliers are rare but critical (e.g., house price errors of ■50L are very costly)	RMSE	Squares the error → heavily penalises large deviations, forcing the model to minimise costly mistakes
Outliers are common / data is noisy (e.g., noisy IoT sensor readings)	MAE	Treats all errors equally; outliers don't dominate the metric
Business cost of large errors is disproportionately high (e.g., HDFC loan defaults: ■1Cr error >> ■1L error)	RMSE	Quadratic penalty aligns metric with asymmetric real-world cost
Median prediction is more meaningful than mean (e.g., income distribution skew)	MAE	MAE corresponds to median-optimal prediction; RMSE to mean-optimal prediction
Comparing across models (same dataset)	Both	RMSE for sensitivity to large errors; MAE for robust overall performance view

PropTech / NoBroker Context: *RMSE is preferred for property price prediction because a ■50 lakh over-valuation is catastrophically worse than a ■5 lakh error for a home buyer taking a 20-year mortgage. RMSE's squared penalty forces the model to avoid such extreme misvaluations.*

Q5 What is Cross-Validation (k-Fold)? Why preferred over single train-test split?

What is Cross-Validation?

Cross-Validation (CV) is a **statistical resampling technique** that evaluates how well a machine learning model generalises to an independent (unseen) dataset. Instead of relying on a single arbitrary train-test split, CV systematically rotates the holdout set to get a robust, low-variance estimate of model performance.

k-Fold Cross-Validation — Step-by-Step:

Step 1: Shuffle the dataset randomly.

Step 2: Divide the dataset into k equal-sized folds (subsets). Typical k = 5 or 10.

Step 3: Repeat k times: Use fold i as the Validation set. Train the model on the remaining k-1 folds. Record the validation score (e.g., RMSE).

Step 4: Final CV Score = Mean of all k validation scores ± Std Dev.

For k = 5 folds:

Fold 1: [TEST][train][train][train][train]

Fold 2: [train][TEST][train][train][train]

Fold 3: [train][train][TEST][train][train]

Fold 4: [train][train][train][TEST][train]

Fold 5: [train][train][train][train][TEST]

■■■

Reproducibility	Results change with random seed.	With shuffle=True and random_state fixed, results are reproducible and stable.

Choosing k:

- **k = 5 or 10** → Standard. Good balance of bias vs variance in the estimate.
- **k = n (Leave-One-Out CV)** → Maximum data use but computationally expensive; high variance.
- **Stratified k-Fold** → Used in classification to maintain class distribution in each fold.

Our Project (Ames Housing): *We used 5-Fold CV on the tuned XGBoost model. CV Log RMSE = 0.1197 ± small std. This aligned closely with the holdout test RMSE ($19,636), confirming that the model generalises well and is not overfitting to the 80% training set.*

Quick Reference Summary — All 5 Topics

Topic	Core Idea	Key Formula / Rule of Thumb
Regression vs Classification	Regression → continuous output. Classification → discrete label.	If output is a number → Regression. If output is a class → Classification.
SLR vs MLR vs Regularised	SLR: 1 feature. MLR: multiple features. Regularised: adds penalty to control complexity.	Ridge: L2 penalty (all coef shrunk). Lasso: L1 penalty (some coef → 0).
Overfitting vs Underfitting	Overfitting: low train error, high test error. Underfitting: high error on both.	Total Error = Bias² + Variance + Noise. Balance via regularisation + CV.
RMSE vs MAE vs R^2	RMSE penalises large errors more. MAE treats all errors equally. R^2 measures explained variance.	RMSE ≥ MAE always. $R^2 = 1$ → perfect. Prefer RMSE when large errors are costly.
Cross-Validation	Rotate holdout fold k times. Average k scores for robust evaluation.	k=5 or k=10 standard. CV Score = Mean ± Std of k validation scores.